

PROJETO FINAL FULLSTACK — PORTAL DE HERÓIS

FIEC - 2026

Disciplina: Linguagem de Programação III & Desenvolvimento Web III

Professor: Gustavo Dias

Data de Lançamento: 01/06/2026.

Prazo de Execução: 23/06

Status: Avaliação Somativa Final (Peso Máximo)

1. VISÃO GERAL DO PROJETO

O desafio final deste semestre consiste em unificar todo o ecossistema de desenvolvimento que aprendemos até aqui. Você irá pegar o conceito do "Portal de Heróis" — que nos acompanhou desde as primeiras linhas de código — e transformá-lo em uma plataforma **Fullstack SaaS Comercial**.

O sistema contará com controle de acesso rigoroso por recrutador, persistência em banco de dados relacional, gerenciamento avançado de estado, regras de negócio baseadas em relacionamentos e uma interface de alta performance com padrão estético profissional.

2. ARQUITETURA E CAMINHO DAS "MIGALHAS" (O QUE DEVE TER NO SISTEMA)

Para que seu projeto seja elegível à nota máxima, sua aplicação deve cumprir obrigatoriamente as rotas, telas e regras de negócio mapeadas abaixo. Recursos extras que acrescentem valor ao produto (como sistemas de níveis, ranking ou logs) são muito bem-vindos e serão considerados como diferencial.

Módulo 1: Sistema de Acesso e Segurança (Autenticação)

O portal é restrito. Para gerenciar heróis, é necessário ser um **Recrutador Credenciado**.

- **Tela de Cadastro de Usuário:** Formulário contendo os campos *Nome Completo*, *E-mail* e *Senha*. No Backend, a senha deve obrigatoriamente passar por um processo de hashing via **BCrypt** antes de salvar no banco de dados.
- **Tela de Login:** Formulário com os campos *E-mail* e *Senha*. Após a validação com sucesso, o servidor deve emitir um **JSON Web Token (JWT)**.

- **Sessão no Cliente:** O Frontend deve capturar o Token JWT enviado pelo servidor, armazená-lo de forma segura (ex: *localStorage*) e utilizá-lo para proteger as rotas internas no React Router.
- **Tela de Configurações de Perfil (/perfil):** Rota interna protegida onde o Recrutador logado pode atualizar seu nome, e-mail e **alterar sua senha**. Para a troca de senha, o Backend deve exigir a "Senha Atual", descriptografar/comparar com o BCrypt e só permitir a gravação se a credencial antiga for válida.

Módulo 2: Tela Dashboard (A Central de Comando)

Esta é a página principal acessada imediatamente após o login. Ela serve como painel de controle da guilda do usuário.

- **Cabeçalho Dinâmico:** Deve exibir com destaque a mensagem: *"Bem-vindo, Recrutador [Nome do Usuário]"* (obtido através do estado global da Context API) e um botão funcional de **Sair (Logout)** que limpa o token e redireciona para a tela de Login.
- **Resumo Estatístico:** No topo da página, exibir três micro-cards com dados agregados (Métricas): "Total de Heróis Recrutados", "Média de Poder da Equipe" e "Guilda Mais Forte".
- **Galeria de Heróis (Listagem Visual):** Cards dinâmicos exibindo os heróis que pertencem **exclusivamente** ao recrutador logado.
 - *Informações no Card:* Nome do herói, Classe (Mago, Guerreiro, Arqueiro ou Ladino), Nível de Poder (0 a 100), o Nome da Guilda associada e um botão visível para **Dispensar Herói (Delete)**.
- **Filtros de Alta Performance:** Um campo de busca por nome que filtra a lista em tempo real na tela. O cálculo do filtro deve utilizar obrigatoriamente o Hook *useMemo* para garantir que a interface não sofra engasgos visuais (*junk*).

Módulo 3: Formação de Equipe e Detalhes (O CRUD na Prática)

- **Tela de Recrutamento (Criação):** Formulário completo para a inserção de novos heróis no banco.
 - *Campos Obrigatórios:* Nome do Herói (mínimo 3 caracteres), Classe (campo Dropdown/Select contendo as opções válidas), Nível de Poder (campo numérico restrito via Zod de 0 a 100) e URL do Avatar (campo de texto validado como URL de imagem para o rosto do herói).
 - *Vínculo Dinâmico:* Um campo Select que lista dinamicamente todas as Guildas cadastradas no banco de dados, forçando o herói a ser criado já vinculado a uma guilda existente.
- **Tela de Detalhes e Evolução (/heroi/:id):** Uma rota dinâmica que captura o ID do herói pela URL.
 - *Exibição:* Dados completos do herói e o seu **Histórico de Missões**.
 - *Atualização (Update):* Um formulário integrado nesta tela que permite editar o Nome, a Classe ou evoluir o Nível de Poder do herói.

Módulo 4: Tabelas de Eventos (O Sistema de Missões)

Para consolidar o conhecimento de bancos de dados relacionais e tabelas de eventos:

- **Histórico de Missões:** Dentro da tela de detalhes de cada herói, deve haver uma seção que consome uma tabela secundária no banco de dados para listar as missões daquele herói.
 - *Campos da Missão:* Descrição (ex: "Limpar a caverna de Goblins"), Status (*Em andamento, Concluída, Falhou*) e a Recompensa em Ouro.
 - **Ação Funcional:** Um botão chamado "Enviar para Missão". Ao ser clicado, ele dispara uma requisição de criação de evento, adicionando dinamicamente uma nova missão ao histórico daquele herói.
-

3. CHECKLIST DE REQUISITOS TÉCNICOS (A STACK)

BACKEND (Node.js + Express + MySQL)

- **Padrão Arquitetural:** Organização estrita do projeto utilizando o padrão de camadas **MVC** (Model-View-Controller) ou separação limpa por rotas, controladores e serviços.
- **Persistência Relacional:** Modelagem correta no banco MySQL contendo tabelas de Entidade (*usuarios, heroes, guildas*) e tabelas de Eventos (*missoes*). Uso obrigatório de Chaves Estrangeiras (*Foreign Keys*) configurando relacionamentos 1:N.
- **Queries Inteligentes:** Uso obrigatório de *INNER JOIN* para retornar os heróis junto com os dados das suas respectivas guildas e criadores em uma única requisição.
- **Proteção Corporativa:** Middlewares customizados para interceptar rotas sensíveis, validando a presença e integridade do Token JWT enviado via Headers (*Authorization: Bearer token*).
- **Validação Rígida:** Proteção do banco de dados contra payloads inválidos utilizando esquemas de validação com a biblioteca **Zod** em todas as rotas de cadastro e inserção.
- **Infraestrutura:** Arquivo *.env* devidamente configurado para isolar credenciais críticas do banco e chaves simétricas. Liberação do **CORS** especificando a URL de produção do seu Frontend.

FRONTEND (React.js + Tailwind CSS v4)

- **Tooling e Estilo:** Aplicação gerada através do **Vite** em JavaScript Puro (Vanilla JS). Estilização desenvolvida com **Tailwind CSS v4** aplicando a identidade visual Premium Dark (predominância de cores como *slate-950, slate-900*, com destaques em texturas vibrantes como *cyan-500, amber-500* ou *pink-500*).
- **Sincronização de Dados:** Uso obrigatório da biblioteca **TanStack Query (React Query)** para todas as requisições HTTP da aplicação (Queries e Mutations). É proibido o uso de *useEffect* soltos para controle de carregamento de APIs.

- **Gerenciamento de Estado Global:** Implementação da **Context API** para centralizar o estado de autenticação (armazenamento do token, dados básicos do usuário logado e controle do fluxo de login/logout).
- **Engenharia de UX e UX Dinâmica:**
 - Tratamento explícito para estados assíncronos: Exibição visual de esqueletos ou mensagens de *Loading...*, tratamento de telas de erro (*Error boundaries*) e placeholders para listas vazias.
 - Mensagens flutuantes (ex: Notificações em Toast) disparadas no sucesso de cada Mutation do TanStack Query (ex: ao recrutar ou dispensar).
 - Uso da biblioteca **Framer Motion** para conferir alta qualidade visual à interface: Animações de transição de páginas no React Router, entrada dos cards na Dashboard utilizando efeitos de cascata (*Stagger*) e animações de feedback na remoção de itens (*AnimatePresence*).

4. ENGENHARIA DE SOFTWARE E DEPLOY

A entrega só será validada caso o ecossistema esteja completamente disponível para acesso na internet:

- **O Repositório (GitHub):** O código-fonte deve estar em um repositório público. O histórico de modificações (*commits*) será rigorosamente avaliado. Repositórios com commits únicos (ex: "projeto pronto") sofrerão penalização severa, uma vez que o versionamento contínuo faz parte da nota.
- **Deploy do Servidor:** O Backend e o banco de dados MySQL devem estar publicados de forma estável em serviços de nuvem como **Render** ou **Railway**.
- **Deploy da Interface:** O Frontend em React deve estar hospedado na plataforma **Vercel**, configurado com pipeline contínuo (CI/CD) integrado ao GitHub.

5. MATRIZ DE AVALIAÇÃO E CRITÉRIOS DE CORREÇÃO

A nota final (0 a 10) será calculada de acordo com o peso de entrega de cada módulo de engenharia:

Módulo Avaliado	Requisito Esperado e Entregáveis Técnicos	Pontuação
Segurança & API (Back)	Funcionamento correto das rotas de Auth, hashing com BCrypt, emissão de JWT, arquitetura MVC estruturada e proteção por Middlewares.	3.0 Pontos

Módulo Avaliado	Requisito Esperado e Entregáveis Técnicos	Pontuação
Persistência (MySQL)	Modelagem íntegra do banco, uso correto de chaves estrangeiras (1:N), integridade dos dados baseados no ID do recrutador e uso de <i>INNER JOIN</i> .	2.0 Pontos
Integração & Estado (Front)	Sincronização de dados via Axios, isolamento de sessão global com Context API e proteção física de caminhos com React Router.	2.5 Pontos
Validação, Performance & UX	Esquemas do Zod blindando Front e Back, tratamento de erros amigáveis, uso do <i>useMemo</i> , TanStack Query e animações com Framer Motion.	1.5 Pontos
Deploy & DevOps (Git)	Aplicação Fullstack completamente pública na internet (Vercel + Render/Railway) e histórico de desenvolvimento fracionado no Git.	1.0 Ponto

6. CRONOGRAMA DAS SEMANAS FINAIS

- **Semana 14 (Hoje):** Lançamento do Manual do Candidato, alinhamento técnico de escopo e validação da modelagem inicial das tabelas do banco de dados no quadro.
- **Semana 15: Plantão de Dúvidas & Laboratório Assistido.** Período totalmente prático em sala de aula. O professor atuará como mentor focado em ajudar a resolver problemas.
- **Semana 16: Bancada de Apresentação Final (Defesa).** Cada desenvolvedor terá até 5 minutos para demonstrar o software rodando ao vivo pelo link da Vercel, realizar o fluxo completo (Cadastro, Login, Inserção de Herói, Envio de Missão e Exclusão) e exibir a organização do repositório no GitHub.